

Experiment 9
CAN Communication

Name: Ashutosh Bajpayee
CDSID: abajpaye



Table of Contents

Sl. No.	Topic	Page No.
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	4
5	Working Principle	4
6	Application Code	5-6
7	Output Image	7
8	Hardware Image	7
9	Appendix	8

Introduction

Controller Area Network (CAN) is a reliable serial communication protocol widely used in automotive and industrial systems for efficient data transfer, allowing multiple microcontrollers to communicate directly without a central host computer. In this experiment, CAN communication is implemented using the Raspberry Pi Pico and the MCP2515 CAN controller module, where two nodes are created—one acting as a transmitter and the other as a receiver—and data is successfully sent from one Pico board to another through the CAN bus and displayed on the serial console.

Scope

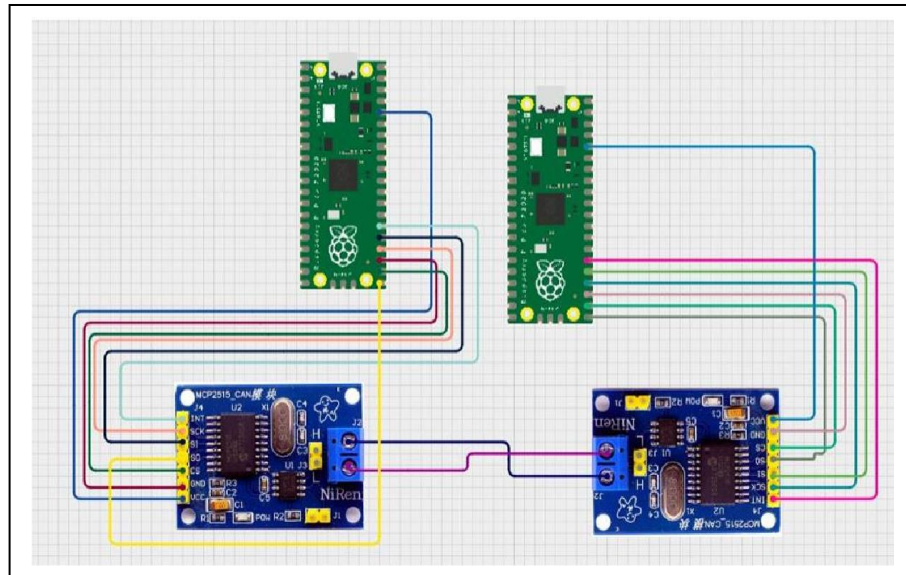
The scope of this experiment is to understand the fundamentals of the CAN protocol, interface the MCP2515 CAN module with the Raspberry Pi Pico using SPI communication, implement communication between transmitter and receiver nodes, and monitor real-time CAN data through a serial terminal.

Prerequisite

Hardware: Raspberry Pi Pico microcontroller (2 units), USB connector (2 units), Breadboard (2 units), MCP2515 CAN module (2 units), 120Ω resistor.

Software: Visual Studio Code, PuTTY serial console, Pico SDK, CMake and GCC toolchain.

Connection Diagram



Working Principle

The MCP2515 module functions as a CAN controller and communicates with the Raspberry Pi Pico using the SPI protocol. The transmitter Pico takes input from the serial terminal (PuTTY) and sends the entered character to the MCP2515 via SPI. The MCP2515 then converts this data into a CAN frame and transmits it over the CAN bus. On the receiving side, the MCP2515 detects the incoming frame and stores it in its RX buffer, after which the receiver Pico reads the data through SPI and displays the received information on the serial terminal.

Application code

Transmitter Code:

```
C Assignment_9_s.c > main()
1  #include "pico/stdlib.h"
2  #include "stdio.h"
3  #include "hardware/spi.h"
4
5  #define SPI_PORT spi0
6  #define PIN_MISO 16
7  #define PIN_CS 17
8  #define PIN_SCK 18
9  #define PIN_MOSI 19
10
11 #define MCP_RESET 0xC0
12 #define MCP_WRITE 0x02
13 #define MCP_RTS_TXB0 0x81
14
15 #define CANCTRL 0x0F
16 #define TXB0SIDH 0x31
17 #define TXB0SIDL 0x32
18 #define TXB0DLC 0x35
19 #define TXB0D0 0x36
20
21 void spi_write_byte(uint8_t address, uint8_t data) {
22     gpio_put(PIN_CS, 0);
23
24     uint8_t buf[3] = {MCP_WRITE, address, data};
25     spi_write_blocking(SPI_PORT, buf, 3);
26
27     gpio_put(PIN_CS, 1);
28 }
29
30 void mcp2515_reset() {
31     gpio_put(PIN_CS, 0);
32
33     uint8_t cmd = MCP_RESET;
34     spi_write_blocking(SPI_PORT, &cmd, 1);
35
36     gpio_put(PIN_CS, 1);
37     sleep_ms(10);
38 }
39
40 void mcp2515_init() {
41     mcp2515_reset();
42
43     spi_write_byte(0x2A, 0x00);
44     spi_write_byte(0x29, 0x90);
45     spi_write_byte(0x28, 0x02);
46
47     spi_write_byte(CANCTRL, 0x00);
48 }
```

```
50 void can_send_message() {
51     spi_write_byte(TXB0SIDH, 0x01);
52     spi_write_byte(TXB0SIDL, 0x00);
53     spi_write_byte(TXB0DLC, 0x01);
54     spi_write_byte(TXB0D0, 0xAA);
55
56     gpio_put(PIN_CS, 0);
57
58     uint8_t cmd = MCP_RTS_TXB0;
59     spi_write_blocking(SPI_PORT, &cmd, 1);
60
61     gpio_put(PIN_CS, 1);
62 }
63
64 int main() {
65     stdio_init_all();
66
67     spi_init(SPI_PORT, 500000);
68
69     gpio_set_function(PIN_SCK, GPIO_FUNC_SPI);
70     gpio_set_function(PIN_MOSI, GPIO_FUNC_SPI);
71     gpio_set_function(PIN_MISO, GPIO_FUNC_SPI);
72
73     gpio_init(PIN_CS);
74     gpio_set_dir(PIN_CS, GPIO_OUT);
75     gpio_put(PIN_CS, 1);
76
77     sleep_ms(2000);
78
79     mcp2515_init();
80
81     printf("CAN Sender Ready...\n");
82
83     while (1) {
84         can_send_message();
85         printf("Message sent!\n");
86         sleep_ms(1000);
87     }
88 }
```

Receiver Code:

```
C Assignment_9_rc > main()
1  #include "pico/stdlib.h"
2  #include "stdio.h"
3  #include "hardware/spi.h"
4
5  #define SPI_PORT spi0
6  #define PIN_MISO 16
7  #define PIN_CS 17
8  #define PIN_SCK 18
9  #define PIN_MOSI 19
10
11 #define MCP_RESET 0xC0
12 #define MCP_WRITE 0x02
13 #define MCP_READ 0x03
14
15 #define CANCTRL 0x0F
16 #define RXB0CTRL 0x60
17 #define RXB0D0 0x66
18 #define CANINTF 0x2C
19
20 // SPI write
21 void spi_write_byte(uint8_t address, uint8_t data) {
22     gpio_put(PIN_CS, 0);
23
24     uint8_t buf[3] = {MCP_WRITE, address, data};
25     spi_write_blocking(SPI_PORT, buf, 3);
26
27     gpio_put(PIN_CS, 1);
28 }
29
30 // SPI read
31 uint8_t spi_read_byte(uint8_t address) {
32     gpio_put(PIN_CS, 0);
33
34     uint8_t tx[3] = {MCP_READ, address, 0x00};
35     uint8_t rx[3];
36
37     spi_write_read_blocking(SPI_PORT, tx, rx, 3);
38
39     gpio_put(PIN_CS, 1);
40
41     return rx[2];
42 }
```

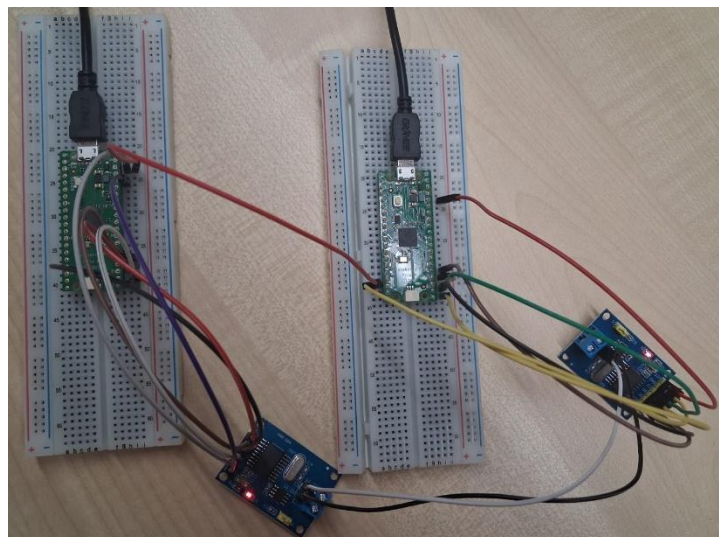
```
C Assignment_9_rc > ...
44 // Reset
45 void mcp2515_reset() {
46     gpio_put(PIN_CS, 0);
47
48     uint8_t cmd = MCP_RESET;
49     spi_write_blocking(SPI_PORT, &cmd, 1);
50
51     gpio_put(PIN_CS, 1);
52
53     sleep_ms(10);
54 }
55
56 // Init
57 void mcp2515_init() {
58     mcp2515_reset();
59
60     // Set CAN bitrate
61     spi_write_byte(0x2A, 0x00);
62     spi_write_byte(0x29, 0x90);
63     spi_write_byte(0x28, 0x02);
64
65     // Enable receive buffer
66     spi_write_byte(RXB0CTRL, 0x00);
67
68     // Normal mode
69     spi_write_byte(CANCTRL, 0x00);
70 }
71
72 // Receive message
73 void can_receive_message() {
74     uint8_t intf = spi_read_byte(CANINTF);
75
76     if (intf & 0x01) {
77         uint8_t data = spi_read_byte(RXB0D0);
78
79         printf("Message Received: 0x%X\n", data);
80
81         // Clear flag
82         spi_write_byte(CANINTF, 0x00);
83     }
84 }
85
86 int main() {
87     stdio_init_all();
88
89     spi_init(SPI_PORT, 500000);
90
91     gpio_set_function(PIN_SCK, GPIO_FUNC_SPI);
92     gpio_set_function(PIN_MOSI, GPIO_FUNC_SPI);
93     gpio_set_function(PIN_MISO, GPIO_FUNC_SPI);
94
95     gpio_init(PIN_CS);
96     gpio_set_dir(PIN_CS, GPIO_OUT);
97     gpio_put(PIN_CS, 1);
98
99     sleep_ms(2000);
100
101     mcp2515_init();
102
103     printf("CAN Receiver Ready...\n");
104
105     while (1) {
106         can_receive_message();
107         sleep_ms(200);
108     }
109 }
```

Output Image

```
COM9 - PuTTY
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
Message sent!
```

```
COM6 - PuTTY
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
Message Received: 0xAA
```

Hardware Image



Appendix

- This experiment demonstrates CAN communication using the Raspberry Pi Pico and MCP2515 CAN controller module.
- The system consists of two nodes configured as transmitter and receiver for data exchange over the CAN bus.
- SPI protocol is used to interface the MCP2515 module with the Pico for sending and receiving data.
- The transmitter sends data frames, which are received and processed by the receiver node.
- The received data is displayed on the serial terminal for monitoring and verification.
- This setup helps in understanding real-time communication in embedded and automotive systems.